

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Lab Assignment 1 (5%) - Indexes

Name	:	TAN ZHI MING
Matric Id	:	A23CS0189

Name	:	NG YU HIN
Matric Id	:	A23CS0148

Name	:	CHAU YING JIA
Matric Id	:	A23CS0213



UNIVERSITI TEKNOLOGI MALAYSIA

Database Programming Assignment 1

GROUP 1

Innovating Solutions

www.utm.my

1. Define Unique Index and explain how it differs from Primary Key.

Unique Index is a database constraint that ensures no two rows can have the same value in the indexed columns. It prevents duplicate values but still allows one NULL value.

Primary Key uniquely identifies each row in a table, is mandatory and does not allow NULL values.

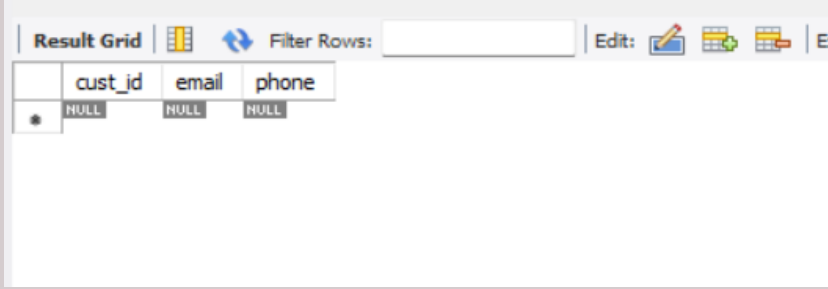
While a table can only have one primary key, it can have multiple unique indexes to enforce uniqueness and improve query performance.

2. List three advantages and three disadvantages of Unique Indexes.

ADVANTAGE	DISADVANTAGE
Ensures data uniqueness and maintains data integrity	Slightly slows down INSERT, UPDATE, and DELETE operations due to uniqueness checks
Improves query performance by speeding up searches and filtering	Consumes additional storage space for the index structure
Reduces data anomalies and prevents duplicate business data	Requires maintenance overhead when data changes frequently

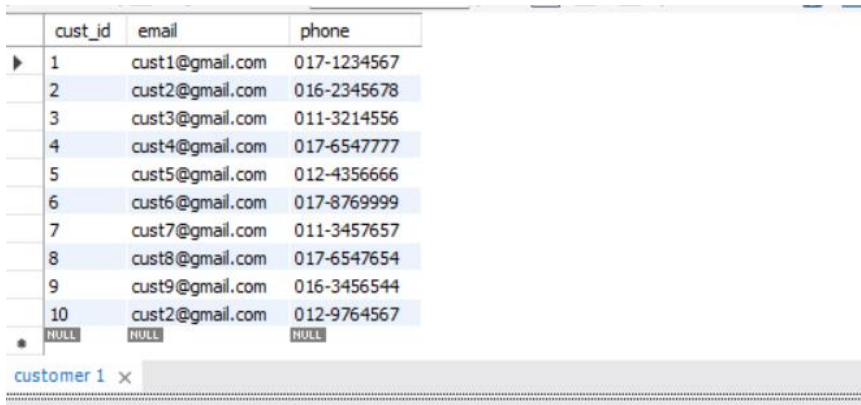
3. Create a table customer

(cust_id INT AUTO_INCREMENT, email VARCHAR(50), phone VARCHAR(15), PRIMARY KEY(cust_id));

CODE	OUTPUT
<pre>CREATE TABLE customer (cust_id INT AUTO_INCREMENT, email VARCHAR(50), phone VARCHAR(15), PRIMARY KEY(cust_id));</pre>	
Create a new table called customer to store records for individuals. Set cust_id as the primary key, which automatically creates a sequential number for every new customer added. Then, make columns to store their email addresses up to 50 characters and phone numbers up to 15 characters.	

4. Insert 10 records

(including a duplicate email) and observe the result.

CODE	OUTPUT																																																
<pre>INSERT INTO customer (email, phone) VALUES ('cust1@gmail.com', '017-1234567'), ('cust2@gmail.com', '016-2345678'), ('cust3@gmail.com', '011-3214556'), ('cust4@gmail.com', '017-6547777'), ('cust5@gmail.com', '012-4356666'), ('cust6@gmail.com', '017-8769999'), ('cust7@gmail.com', '011-3457657'), ('cust8@gmail.com', '017-6547654'), ('cust9@gmail.com', '016-3456544'), ('cust2@gmail.com', '012-9764567'); SELECT * FROM customer;</pre>	 <table><thead><tr><th></th><th>cust_id</th><th>email</th><th>phone</th></tr></thead><tbody><tr><td>▶</td><td>1</td><td>cust1@gmail.com</td><td>017-1234567</td></tr><tr><td></td><td>2</td><td>cust2@gmail.com</td><td>016-2345678</td></tr><tr><td></td><td>3</td><td>cust3@gmail.com</td><td>011-3214556</td></tr><tr><td></td><td>4</td><td>cust4@gmail.com</td><td>017-6547777</td></tr><tr><td></td><td>5</td><td>cust5@gmail.com</td><td>012-4356666</td></tr><tr><td></td><td>6</td><td>cust6@gmail.com</td><td>017-8769999</td></tr><tr><td></td><td>7</td><td>cust7@gmail.com</td><td>011-3457657</td></tr><tr><td></td><td>8</td><td>cust8@gmail.com</td><td>017-6547654</td></tr><tr><td></td><td>9</td><td>cust9@gmail.com</td><td>016-3456544</td></tr><tr><td></td><td>10</td><td>cust2@gmail.com</td><td>012-9764567</td></tr><tr><td>*</td><td>NULL</td><td>NULL</td><td>NULL</td></tr></tbody></table> customer 1 x		cust_id	email	phone	▶	1	cust1@gmail.com	017-1234567		2	cust2@gmail.com	016-2345678		3	cust3@gmail.com	011-3214556		4	cust4@gmail.com	017-6547777		5	cust5@gmail.com	012-4356666		6	cust6@gmail.com	017-8769999		7	cust7@gmail.com	011-3457657		8	cust8@gmail.com	017-6547654		9	cust9@gmail.com	016-3456544		10	cust2@gmail.com	012-9764567	*	NULL	NULL	NULL
	cust_id	email	phone																																														
▶	1	cust1@gmail.com	017-1234567																																														
	2	cust2@gmail.com	016-2345678																																														
	3	cust3@gmail.com	011-3214556																																														
	4	cust4@gmail.com	017-6547777																																														
	5	cust5@gmail.com	012-4356666																																														
	6	cust6@gmail.com	017-8769999																																														
	7	cust7@gmail.com	011-3457657																																														
	8	cust8@gmail.com	017-6547654																																														
	9	cust9@gmail.com	016-3456544																																														
	10	cust2@gmail.com	012-9764567																																														
*	NULL	NULL	NULL																																														
This code inserts ten new customer records into the customer table, providing email and phone numbers, while the database automatically assigns a numbering to every row.																																																	

5. Add a unique index on email

```
ALTER TABLE customer ADD UNIQUE idx_email (email);
```

CODE	EXPLANATION
ALTER TABLE customer ADD UNIQUE idx_email (email);	This script tries to add a UNIQUE index on the email column in the customer table, enforcing that no two customers can share the same email.
DELETE FROM customer WHERE cust_id = 10;	It then deletes the row where cust_id = 10 and attempts to add the same unique index again.
ALTER TABLE customer ADD UNIQUE idx_email (email);	This demonstrates that creating a UNIQUE constraint will fail if duplicate emails exist , so deleting conflicting rows allows the constraint to be successfully added.

OUTPUT:

✖	17 16:50:34 ALTER TABLE customer ADD UNIQUE idx_email (email)	Error Code: 1062. Duplicate entry 'cust2@gmail.com' for key 'customer.idx_email'	0.156 sec
✔	18 16:50:40 DELETE FROM customer WHERE cust_id = 10	1 row(s) affected	0.015 sec
✔	19 16:50:44 SELECT * FROM customer LIMIT 0, 10000	9 row(s) returned	0.000 sec / 0.000 sec
✔	20 16:50:44 ALTER TABLE customer ADD UNIQUE idx_email (email)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

6. Try inserting a duplicate email again and record the error

CODE: INSERT INTO customer (email, phone)
 VALUES ('cust9@gmail.com', '019-9998888');

OUTPUT:

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 15	16:50:30	INSERT INTO customer (email, phone) VALUES ('cust1@gmail.com', '017-1234567'), ('cust2@gmail.com', '0...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
✓ 16	16:50:30	SELECT * FROM customer LIMIT 0, 10000	10 row(s) returned	0.000 sec / 0.000 sec
✗ 17	16:50:34	ALTER TABLE customer ADD UNIQUE idx_email (email)	Error Code: 1062. Duplicate entry 'cust2@gmail.com' for key 'customer.idx_email'	0.156 sec
✓ 18	16:50:40	DELETE FROM customer WHERE cust_id = 10	1 row(s) affected	0.015 sec
✓ 19	16:50:44	SELECT * FROM customer LIMIT 0, 10000	9 row(s) returned	0.000 sec / 0.000 sec
✓ 20	16:50:44	ALTER TABLE customer ADD UNIQUE idx_email (email)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
✗ 21	16:53:17	INSERT INTO customer (email, phone) VALUES ('cust9@gmail.com', '019-9998888')	Error Code: 1062. Duplicate entry 'cust9@gmail.com' for key 'customer.idx_email'	0.000 sec

EXPLANATION:

Adds one new row to the customer table with email 'cust9@gmail.com' and phone number '019-9998888', automatically assigning it the next available ID.

However, it will show an error message showing that there is a duplicate email address as we have set the unique index that ensure there is no duplicate email in the table.

7. Use SHOW INDEX FROM customer;

and explain the output

CODE: `SELECT INDEX FROM customer;`

OUTPUT:

Result Grid Filter Rows: Export: Wrap Cell Content:															
	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
▶	customer	0	PRIMARY	1	cust_id	A	0	HULL	HULL		BTREE			YES	HULL
	customer	0	idx_email	1	email	A	9	HULL	HULL	YES	BTREE			YES	HULL

Result 5 x Read Only

EXPLANATION:

The result shows two indexes on the customer table.

The first is the PRIMARY key on cust_id, which uniquely identifies each row.

The second is idx_email, a UNIQUE index on the email column, ensuring no two customers share the same email.

Both indexes use BTREE, which allows fast lookup and efficient searching.

8. Describe how a unique index improves lookup performance

- A unique index uses a B-tree (or similar) structure to locate rows efficiently without scanning the full table.
- Lookups operate in roughly **$O(\log n)$** time, improving performance compared to full table scans.
- Because values are guaranteed to be unique, the database can stop searching as soon as a match is found.
- This reduces disk I/O and CPU work during lookups.
- Unique indexes often perform better than no index and can sometimes outperform non-unique indexes due to early termination of the search.

9. Compare query performance

```
SELECT * FROM customer WHERE email='user1@gmail.com';
```

OUTPUT:

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	customer	NULL	const	idx_email	idx_email	203	const	1	100.00	NULL

With index

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	customer	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where

Without index

CODE:

```
EXPLAIN SELECT * FROM customer WHERE
email='cust9@gmail.com';
DROP INDEX idx_email ON customer;
EXPLAIN SELECT * FROM customer WHERE
email='cust7@gmail.com';
```

EXPLANATION:

With the index, MySQL uses an efficient index lookup and scans only one row, resulting in much faster query performance. Without the index, MySQL performs a full table scan, checking all rows, which is slower and inefficient. This demonstrates how indexes significantly improve query speed by reducing the amount of data MySQL needs to scan.

10. Reflect: When is a unique index not appropriate for a column?

- Not suitable for columns that naturally contain duplicates (e.g., city, country, status, gender).
- Inefficient for columns that change frequently, since maintaining the index slows INSERT/UPDATE/DELETE operations.
- Provides limited value for optional columns or those that often contain NULLs.
- Unnecessary when the column is rarely used in search or lookup queries.
- Ineffective for attributes that are not logically unique (e.g., favorite color, product category).
- Can hinder bulk insert performance because each row requires a uniqueness check.

THANK YOU



univteknologimalaysia



utm.my



utmofficial